

# Harness框架对比

## 背景

The goal of a harness is to mold the inherently spiky intelligence of a model for tasks we care about.

**Harness Engineering** is about systems, you're building tooling around the model to optimize goals like task performance, token efficiency, latency, etc. Design decisions include the system prompt, tool choice, and execution flow.

## 第一轮

### 基本信息

| 框架                 | 是否开源                                                                                | 许可证                     | 语言             | Harness<br>可定制性            | System<br>Prompt可控<br>性                        | Tool<br>Orchestra<br>tion可配性                    | 大模型支<br>持丰富度                  |
|--------------------|-------------------------------------------------------------------------------------|-------------------------|----------------|----------------------------|------------------------------------------------|-------------------------------------------------|-------------------------------|
| Claude<br>Code CLI |  | Proprietar<br>y License | TpyeS<br>cript | ★ 低<br>几乎不可<br>定制，黑<br>盒封装 | ★ 极低<br>System<br>Prompt完全<br>不可见/不可<br>改，官方控制 | ★★★ 较固<br>化<br>工具集由<br>Anthropic<br>预设，扩<br>展受限 | ★ 极窄<br>仅支持<br>Claude系<br>列模型 |

|                         |   |               |        |         |                                                                                                                      |              |                                                   |
|-------------------------|---|---------------|--------|---------|----------------------------------------------------------------------------------------------------------------------|--------------|---------------------------------------------------|
| OpenAI<br>Codex CLI     | ✓ | Apache<br>2.0 | Rust   | ★★★★ 中等 | ★★★★ 有限<br>(官方计划增强)<br><br>system<br>prompt<br>是"leaked"状态，官方未公开支持自定义。(注: <code>Prompt engineering</code> 能力在社区提案阶段) | ★★★★ 较<br>固化 | ★★★★ 主<br>要OpenAI<br><br>深度绑定<br>OpenAI生态         |
| Google<br>Gemini<br>CLI | ✓ | Apache<br>2.0 | Python | ★★★★ 中等 | ★★★★ 有限<br><br>通过<br>Extension和<br>GEMINI.md配置，但核心<br>prompt不可触及。                                                    | ★★★★ 较<br>固化 | ★★★★ 主<br>要Google<br><br>强以来<br>Google<br>Cloud生态 |
| OpenCode                | ✓ | MIT           | Go     | ★★★★ 高  | ★★★★ 较开<br>放<br><br>支持通过<br>hooks和<br>protocol<br>prompt配置，<br>但核心agent<br>loop是用Go<br>写的。                           | ★★★★ 可配置     | ★★★★ 支持<br>75+模型                                  |

|            |   |            |        |                                                                     |                                                                                                |                                                                        |                                                                        |
|------------|---|------------|--------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------|
| Aider      | ✓ | Apache 2.0 | Python | ★★★★★<br>极高                                                         | ★★★★★ 完<br>全开放<br><br>可直接编辑<br>Prompt模板<br>(aider/coder<br>s/base_coder<br>.py当中的<br>prompt模板) | ★★★★★<br>可深度裁<br>剪                                                     | ★★★★★<br>★支持本<br>地/多模型                                                 |
| Openhands  | ✓ | MIT        | Python | ★★★★★<br>极高                                                         | ★★★★★ 完<br>全开放                                                                                 | ★★★★★<br>可深度裁<br>剪                                                     | ★★★★★<br>★支持本<br>地/多模型                                                 |
| CrewAI     | ✓ | MIT        | Python | ★★★★★<br>极高<br>YAML驱动<br>角色/任务<br>配置，支<br>持自定义<br>Agent<br>Pipeline | ★★★★★ 完<br>全开放<br>每个Agent的<br>system<br>prompt可独立<br>定制，支持角<br>色/目标/背景<br>注入                   | ★★★★★<br>可深度裁<br>剪<br>支持自定义<br>Tool/Plugin<br>，兼容<br>LangChain<br>工具生态 | ★★★★★<br>★支持本<br>地/多模型<br>通过<br>LiteLLM<br>支持<br>100+模型                |
| DeepAgents | ✓ | MIT        | Python | ★★★★★<br>极高<br>支持多<br>Agent拓扑<br>自定义，<br>节点/流程<br>可深度配<br>置         | ★★★ 完全开放<br>每层Agent<br>prompt均可自<br>定义，支持多<br>级prompt注入                                        | ★★★ 可<br>配置<br>支持工具<br>注册机<br>制，但生<br>态成熟度<br>弱于<br>CrewAI             | ★★★★★<br>支持主流<br>模型<br>优先适配<br>国内LLM<br>生态<br>(DeepS<br>eek/Qwen<br>等) |

|      |   |     |        |                                                             |                                                                                                                       |                                                                                       |                                                                   |
|------|---|-----|--------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| DSPy | ✓ | MIT | Python | ★★★★★<br>极高<br>以编程方<br>式定义模<br>块化<br>Pipeline,<br>完全可组<br>合 | ★★★★★ 完<br>全开放（编程<br>范式）<br>Prompt非手写<br>而是自动优化<br>生成，通过<br>Optimizer/Tel<br>eprompter自<br>动调优，颠覆<br>传统prompt控<br>制方式 | ★★★★★<br>可配置<br>支持<br>ReAct/Tool<br>调用模块，<br>但偏向研究<br>向，工程<br>Tool链集成<br>弱于<br>CrewAI | ★★★★★<br>★支持本<br>地/多模型<br>通过<br>LiteLLM/<br>自定义LM<br>接口支持<br>主流模型 |
|------|---|-----|--------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------|

## 可开发性

### Claude code

明确禁止逆向工程、代码修改和再分发。

Claude Code 通过 npm 分发（`@anthropic-ai/claude-code`），但 Anthropic 对源代码进行了混淆处理（obfuscation）以增加逆向工程难度：

1. 无法直接获取清晰的源代码 —— 本地安装的是编译/混淆后的代码
2. 法律风险 —— 曾有开发者成功反混淆并公开了 Claude Code 的源代码，Anthropic 随即提交了 DMCA 投诉要求下架

但目前因为此前的0331代码泄漏事件，网上存在大亮Claude Code源码。

### Aider

Aier的架构设计对二次开发比较友好：

1. repo map机制：使用 `tree-sitter` 构建代码库地图，通过图排名算法动态选择最相关的上下文。可以基于此机制构建业务场景中的订单状态图、商家产能图等，形成整体“业务实体关系图”。
2. 双模式架构：Architect Mode + Code Mode，将 规划 和 执行 拆分维护。
3. 完全开源Python：核心逻辑在 `aider/coders/` 目录，你可以直接：
  - a. 修改 `base_coder.py` 中的system prompt模板
  - b. 精简tool集合（移除web搜索等无关工具，保留文件操作、代码执行）
  - c. 注入业务领域知识（外卖调度规则、时效约束）

## 第二轮

### Aider v.s. OpenHands

| 维度                  | Aider                                                                             | OpenHands                                                                                       |
|---------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 开源项目                | <a href="https://github.com/Aider-AI/aider">https://github.com/Aider-AI/aider</a> | <a href="https://github.com/AI-Hands-AI/OpenHands">https://github.com/AI-Hands-AI/OpenHands</a> |
| 定位                  | programmable coding copilot                                                       | 通用AI agent平台（可扩展到非代码领域）                                                                         |
| 是否具备agent runtime   | ❌                                                                                 | ✅                                                                                               |
| 自定义Agent            | 不支持，架构固定为coding assistant                                                         | 支持，SDK提供完整的custom agent开发                                                                       |
| 自定义Tool             | 不支持。tool/function calling 是开放issue，尚未实现                                           | 原生支持。Action-Execution-Observation模式，可注册自定义tool                                                  |
| system prompt       | 可通过yaml配置有限调整，但核心prompt围绕代码编辑                                                     | 完全自定义，支持per-agent独立prompt                                                                       |
| Harness Engineering | 弱。主要依赖Git Diff评估，无复杂任务追踪。                                                         | 原生支持。包含评估框架（Event Stream）和任务轨迹回放，可以有效对agent runtime过程中的各个环节进行追踪、评估、优化。                          |
| MCP协议               | 社区提议中，未实现                                                                         | 已支持MCP集成                                                                                        |
| LLM支持               | 75+ provider（via litellm）                                                         | 100+ LLM，支持自训练模型，支持无function calling的模型fallback                                                 |
| Python生态位           | Python编写，但API不开放为SDK                                                              | Python SDK，适合你的技术栈                                                                              |
| 部署成本                | 轻量，CLI直接运行                                                                        | 需要Docker sandbox，相对重一些                                                                          |
| 开发成本                | 低，开箱即用。但改造成业务agent的成本极高，等于重写                                                      | 中等。需要学习SDK，需要投入人力研发，但自定义agent是一等公民。                                                             |
| 维护成本                | 低                                                                                 | 中等。社区活跃，但版本迭代需跟进                                                                                |

# 结论

排除 Aider，暂时优选 OpenHands。

Aider就是一个端用户工具，对标 Claude Code，不太适合作为 agent 架构开发的基座。

Openhands 虽然偏好 coding 场景，尤其是对 SWE 做了很强的绑定，但是具备优秀的 evaluation harness。不过它的整体架构可能不太匹配 Claude Code 现有模式，改造适配成本比较大。可以再观察观察有没有更好的架构方案选择。

## 第三轮

### OpenHands v.s. DeepAgents

| 维度          | DeepAgents                                                                                          | OpenHands                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| github      | <a href="https://github.com/langchain-ai/deepagents">https://github.com/langchain-ai/deepagents</a> | <a href="https://github.com/OpenHands/OpenHands">https://github.com/OpenHands/OpenHands</a> |
| 流行度         | ★ star: 70.7k                                                                                       | ★ star: 19.6k                                                                               |
| 特点          | build on LangGraph。基于DAG的状态机，强调任务的规划、分界、持久化。                                                        | 基于 Event Stream 异步架构，强调 Agent 与环境的强交互                                                       |
| 模型支持        | 全部                                                                                                  | 全部                                                                                          |
| 开发语言        | Python                                                                                              | Python                                                                                      |
| 可定制开发       | 极致的流程/策略定制。如果你需要 Agent在遵循严格的业务suffix，可以通过它的 DAG 很好实现。                                               | 极致的工具/环境定制。                                                                                 |
| Harness能力   | 偏向节点级 trace，需要外接 LangSmith/Langfuse 来精确看到哪一步出错了                                                     | 内置端到端交互轨迹回放，可以实现轨迹数据收集                                                                      |
| Thinking兼容性 | 极高。<br>原生支持 Planning Tool (wrtie_todos)，将思考过程显性化为待办清单。                                              | 中。<br>主要通过解析器提取，侧重于将思考转化为Action。                                                            |

|          |                                                                                                                                                 |                                                                                                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 开发/运维成本  | <ul style="list-style-type: none"> <li>开发成本中等，基于LangGraph生态，有丰富的文档参考</li> </ul>                                                                 | <ul style="list-style-type: none"> <li>开发成本略高，需理解其复杂的事件流；</li> <li>每个 Agent Session 需要一个 Docker 容器，资源开销比较大</li> <li>对齐了 SWE-bench，自定义 Action/Observation 时需要剥离 SWE的特化逻辑。</li> </ul> |
| 多agent并行 | ✓                                                                                                                                               | ✓                                                                                                                                                                                   |
| 训练接入成本   | 高：需要外接 LangChain 生态的其他组件，自建。                                                                                                                    | 中高：有轨迹数据收集能力，但需要数据适配后接入训练 pipeline。                                                                                                                                                 |
| 评测接入成本   | 中：可以通过 Langfuse 作为媒介对接 DeepEval 实现闭环。                                                                                                           | 高：需自建。                                                                                                                                                                              |
| 参考       | <ul style="list-style-type: none"> <li><a href="https://docs.langchain.com">docs.langchain.com</a></li> <li><a href="#">Examples</a></li> </ul> |                                                                                                                                                                                     |
| 合适场景     | 如果是“对业务逻辑的严密控制”，用少量代码把垂域模型和业务工具串联起来，并通过 LangGraph 提供的状态控制，确保模型的hooks 逻辑。                                                                        | agent需要再复杂的仿真环境中不断测试、改代码、运行脚本。                                                                                                                                                      |

## 业务应用条件分析

### Harness Engineering

#### DeepAgents

- system prompt**：原生支持 AGENT.md / MEMORY.md / SKILL.md 分层 prompt 管理，可以精细控制 agent的行为边界和业务知识注入。
- tool orchestration**：基于 LangGraph DAG 结构，每个节点是一个tool/子agent调用，支持条件分支、并行、循环。适合营销活动推荐这种复杂的决策流程实现。
- multi-agent**：内置 supervisor/handoff 模式，可以拆分多 agent 协同。
- mcp**：原生支持 MCP 协议
- context engineering**：内置上下文裁剪、压缩机制，对长对话/多工具场景友好。

#### OpenHands

1. Agent 抽象层 (reset() + step() + action) 足够灵活, 但默认动作空间是参考 SWE 情景的。
2. Docker 沙箱执行环境启动
3. 没有内置 prompt 分层管理结构, 单一字符串维护
4. tool orchestration: step() 手搓, 没有DeepAgents的可视化状态图

## LLM training

### DeepAgents

1. 纯 Harness 框架, 与 LLM 的交互是纯 API 调用;
2. 轨迹数据需要通过外接 LangChain 其他组件 (LangSmith/Langfuse) 收集, 需要自行搭建 data pipeline。

### OpenHands

1. 有可参考的example: trajecotory collection → rejection sampling → SFT → OpenHands\_LM\_32B (based on Qwen2.5-Coder-32B);
2. agent 运行时自动收集轨迹数据 (state, action), 可直接用于 SFT 数据集合成;
3. 但轨迹数据格式和 SWE-bench 强绑定, 需要根据需要做适配性转换。

## 结论

排除 OpenHands, 暂时优选 DeepAgents。

考量:

1. **Harness开发层**: DeepAgent 的 LangGgraph 编排 + 分层 Prompt + MCP 工具 的实现成本远低于 OpenHands在 step() 手搓。
2. **LLM训练层面**: 两个框架都不提供开箱即用的训练 pipeline, 都需要考虑数据标准化后接入我们的 nebula 集群走 mdl 任务训练。OpenHands 虽然省去了轨迹数据收集 (原生) 开发工作, 但数据格式是 SWE 特化的, 需要额外的标准化转换。
3. **测试层面**: DeepAgents 直接对接我在测评层构想的架构模式 (Langfuse + DeepEval) 会比较丝滑。

基于DeepAgents的具体落地路径:

DeepAgents (harness)

└—— AGENT.md → 外卖营销 agent 的角色定义和行为边界



- SKILL.md → 推广包推荐、满减设计、定价协同等技能
- LangGraph → 多步决策状态图编排
- MCP Tools → 业务 API（商家数据、活动数据、定价引擎）
- Langfuse → 轨迹收集 + 评测

#### 训练 Pipeline

- Langfuse traces → SFT 数据集构建
- 基座模型（Qwen）
- 训练框架（TuningFactory / ROLL）
- 评测：“固定 harness + DeepEval” 对比 checkpoint

## 第四轮

| 维度     | DeepAgents                                                                                          | DSPy                                                                                  | CrewAI                                                                                | ForgeCode                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| github | <a href="https://github.com/langchain-ai/deepagents">https://github.com/langchain-ai/deepagents</a> | <a href="https://github.com/stanfordnlp/dspy">https://github.com/stanfordnlp/dspy</a> | <a href="https://github.com/crewaiinc/crewai">https://github.com/crewaiinc/crewai</a> | <a href="https://github.com/antinomyhq/forgecode">https://github.com/antinomyhq/forgecode</a> |
| 核心定位   | Agent Harness runtime                                                                               | LLM pipeline 编程 + 框架优化                                                                | 多 Agent 协作编排平台                                                                        | 终端原生的 AI coding agent                                                                         |
| 设计逻辑   | Context Engineering + 规划/记忆/隔离                                                                      | Prompting, not prompting                                                              | 角色扮演式多 agent 协作                                                                       | World's #1 Coding Harness (terminal-benchmark 排行NO.2)                                         |
| 许可证    | MIT                                                                                                 | MIT                                                                                   | Apache 2.0 + 商业版                                                                      | Apache 2.0                                                                                    |

|                    |                                                                                                         |                                                                                                                                                                                                                 |                                                                                                                                   |                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Harness 开发与优化      | <ol style="list-style-type: none"> <li>1. 自建测试框架，手动/自动迭代</li> <li>2. 有 LangGraph 状态图呈现，完全可视化</li> </ol> | <ol style="list-style-type: none"> <li>1. 原生 Evaluate 模块 + 自定义 metric</li> <li>2. MIPROv2 / BootstrapFe wShot 对 prompt 自动寻优</li> <li>3. 模块化设计天然可测，支持 assert-based metric</li> <li>4. 可通过 MLflow 观测结果</li> </ol> | <ul style="list-style-type: none"> <li>• 抽象层级过高，虽然让搭建 agent 变得非常简单，但同样导致底层可定制化很有限</li> <li>• 适合 prototype 和 demo 的快速落地</li> </ul> | 不支持 SKILL.md 提供的软进化方式                                                                                              |
| system prompt      | 通过 SKILL.md + context engineering 定制                                                                    | Signature/Module 抽象，prompt 由框架自动生成和优化                                                                                                                                                                           | 通过 Agent role/goal/backstory 定制                                                                                                   | forge.yaml 配置 + AGENT.md                                                                                           |
| Multi-Agent        | supervisor/hand off 模式实现                                                                                | 支持，但效果不确定                                                                                                                                                                                                       | 最佳，是比较成熟的多 agent 协作框架。                                                                                                            | forge.yaml 多 agent workflow                                                                                        |
| tool orchestration | 基于 LangGraph 工具注册                                                                                       | dspy.Tool + ReAct module 程序化实现                                                                                                                                                                                  | 装饰器定义，不够灵活                                                                                                                        | <ul style="list-style-type: none"> <li>• 通过 forge.yaml pre-agent 配置</li> <li>• tool 需要通过 shell/MCP 间接调用</li> </ul> |

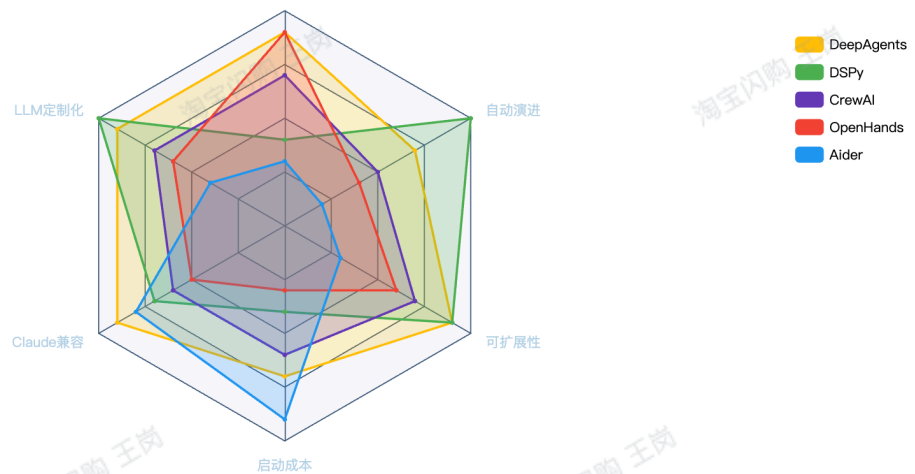
|         |                                                             |                                                                                                                                                                                                                                                                                            |                                      |           |
|---------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|-----------|
| 训练数据可得性 | 无，需外接 <code>Lang fuse</code>                                | <p>BootstrapFinetune 自动收集轨迹并编译为 SFT 数据</p> <p>一旦定义好 Signature（输入输出格式）和 metric（测评标准），可以自动完成：</p> <ol style="list-style-type: none"> <li>1. 用 teacher 模型（如 Claude-Opus-4.6）跑我的 Agent Pipeline 收集轨迹数据；</li> <li>2. 对轨迹数据做 RS 筛选出高质量数据；</li> <li>3. 将数据编译成 SFT 训练集，用于 fine-tune 模型。</li> </ol> | 无                                    | 无         |
| 学习曲线    | <p>中。继承 LangGraph 逻辑。</p> <p>但整个架构参考 <b>Claude Code</b></p> | <p>陡（Signature/Module/Optimizer 范式转换）</p> <p>和 claude code 架构没有直接关系</p>                                                                                                                                                                                                                    | 低（直觉式 API）但一些架构理念甚至和 claude code 冲突。 | 高。Rust 编译 |
| PoC 实现性 | 快                                                           | 中（需设计 Module 结构）                                                                                                                                                                                                                                                                           | 快                                    | 快         |
| 运行成本    | 中                                                           | 低                                                                                                                                                                                                                                                                                          | 高                                    | 低         |

## 参考

## 学术根基扎实

- [https://dspy.ai/tutorials/customer\\_service\\_agent/](https://dspy.ai/tutorials/customer_service_agent/)
- [https://dspy.ai/tutorials/classification\\_finetuning/](https://dspy.ai/tutorials/classification_finetuning/)

Agent框架多维度对比评估



## 结论

采用DeepAgents + DSPy 的组合模式：

- DeepAgents 做 Harness 编排 (LangGraph 状态图 + MCP + Langfuse)
- DSPy 做 harness engineering 寻优 和 训练 pipeline (BootstrapFinetune 消费 Langfuse 导出的trace数据, MIPROv2 自动寻优 prompt)
- DSPy 可以支持在线数据 (teacher) 到离线训练 (student) 的完整链路: Evaluate → BootstrapFewShot → BootstrapFinetune

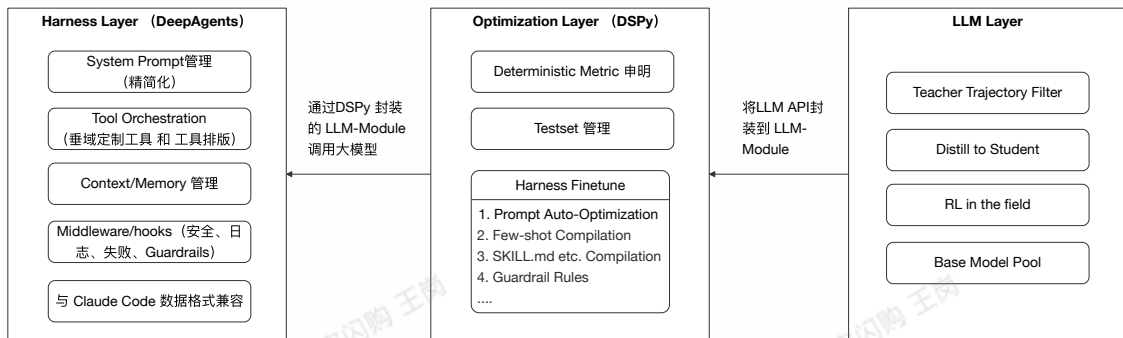
基于上述的多轮分析：

| 维度                    | DeepAgents | DSPy  | CrewAI | OpenHands | Aider |
|-----------------------|------------|-------|--------|-----------|-------|
| Harness Engineering   | ★★★★★      | ★☆☆☆☆ | ★★☆☆☆  | ★★★★☆     | ★★★★☆ |
| Tool Orchestration 定制 | ★★★★★      | ★★☆☆☆ | ★★☆☆☆  | ★★★★☆     | ★★★★☆ |
| System Prompt 精细控制    | ★★★★★      | ★★★★★ | ★★☆☆☆  | ★★★★☆     | ★★★★☆ |
| Evaluation Pipeline   | ★★★☆☆      | ★★★★★ | ★☆☆☆☆  | ★★★★★     | ★★★★☆ |
| LLM训练/微调支持            | ★☆☆☆☆      | ★★★★★ | ★☆☆☆☆  | ★☆☆☆☆     | ★★★★☆ |
| Auto-Evolution能力      | ★★★☆☆      | ★★★★★ | ★☆☆☆☆  | ★★★★☆     | ★★★★☆ |
| 非 Coding域 适配性         | ★★★★☆      | ★★★★★ | ★★★★★  | ★★★★☆     | ★★★★☆ |
| Claude Code兼容         | ★★★★★      | ★★☆☆☆ | ★☆☆☆☆  | ★★★★☆     | ★★★★☆ |
| 启动成本（低=好）             | ★★★☆☆      | ★★☆☆☆ | ★★★★★  | ★☆☆☆☆     | ★★★★★ |
| 社区成熟度                 | ★★★☆☆      | ★★★★☆ | ★★★★★  | ★★★★☆     | ★★★★☆ |

定位我的场景需求分解后的最佳方案：

| 需求                                                        |            |
|-----------------------------------------------------------|------------|
| Agent Harness                                             | DeepAgents |
| Harness Finetune（based on deterministic metric + testset） | DSPy       |
| LLM Finetune                                              | DSPy       |
| Claude Code架构兼容 + 数据迁移                                    | DeepAgents |
| 外卖业务场景适配                                                  | DeepAgents |

所以我计划采用 DeepAgents + DSPy 的双层结构来实现整个“外卖商家营销活动推荐和配置智能代理”：



Harness层和Optimization层通过LLM调用层解耦 —— DeepAgents调LLM的部分用 DSPy Module封装，实现harness和optimization的无缝衔接。

补充说明：LangChain 团队在 DeepAgents 上通过 SKILLS 也实现了automated trace analysis，可以参考该docs： <https://docs.langchain.com/langsmith/skills>

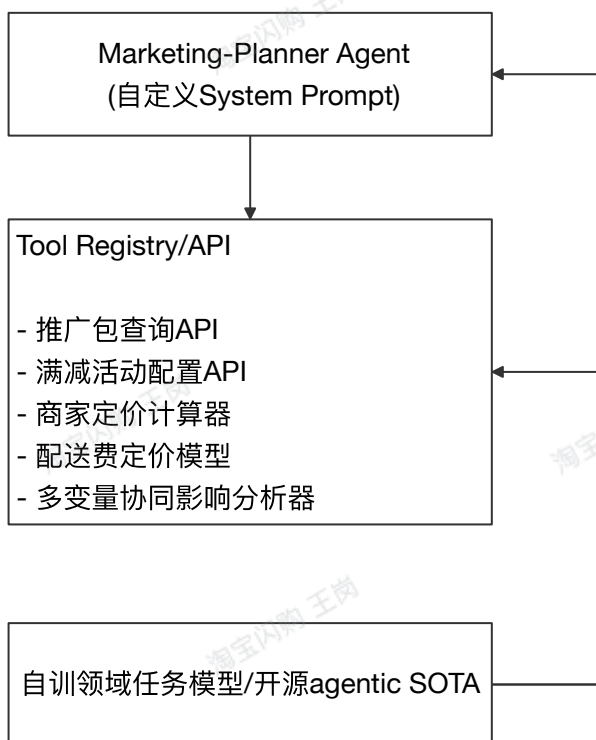
#### schedule：

| 时间       | 行动项                                          |
|----------|----------------------------------------------|
| week1    | Claude Code 和 DeepAgents 原生 Harness在垂域场景上的摸底 |
| week 2-3 | DeepAgents框架学习 和 harness prototype 设计开发      |
| week 4-5 | DSPy 框架学习 和 auto-harness 实践，驱动 harness 改进    |
| week 6-7 | DeepAgents 和 DSPy 集成部署，检验场景效果                |
| ...      | ...                                          |

# 结合业务需求的详细分析

## 场景实现模拟

一下是“营销活动推荐和配置”的大体实现结构，我们从“工程pipeline实现”和“业务场景实现”2个角度说明一下：



## 工程实现

### 1.1 功能点适配

Phase 1: 固定Harness + 通用LLM（如Claude/GPT）+ 自定义tool，快速验证agent效果

Eval

Phase 2: 收集agent交互数据，训练领域专用LLM

Sync

Phase 3: 用自训练LLM替换通用LLM，精简system prompt，降低推理成本

从harness engineering能力看，整个开发过程涉及到：

1. offline eval
2. policy search
3. reward modeling
4. mutli agent routing
5. simulation

DeepAgents、openhands等开源架构都是天然支持：

- agent lifecycle
- environment
- multi agent
- sandbox

所以我们可以固定Agentic Capability SOTA模型（当前推荐直接用claude opus 4.6），记录下所有失败的业务决策轨迹；基于轨迹的分析找出当前通用Harness不适配的部分对Tool Orchestration做改造，例如引入State Machine约束，确保agent在给出推荐方案前都必须调用一次“成本预估工具”。

## 1.2 成本压缩

| 优点 | 说明 |
|----|----|
|----|----|



|                             |                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>System Prompt精简</b>      | Claude Code原生的Prompt包含了大亮针对通用编程任务的约束。在我们的业务场景下，可以剔除所有关于“文件编辑、Git操作、Lint检查”等相关的描述。                                      |
| <b>Tool Orchestration精选</b> | 可以保留3-5个核心工具，例如 <code>商家活动信息获取</code> 、 <code>价格更新</code> 等，将部分涉及到复杂SQL逻辑的数据获取操作固化为标准接口并将数据落入file system中，交给LLM自主应用决策。 |
| <b>规模化应用</b>                | 如果未来普及“营销活动推荐和配置”agent，每推荐一次营销方案可能需要消耗数万token，对于大规模应用直接接入claude code类似产品，API费用巨大。                                      |

## 业务约束

“营销活动推荐和配置”的场景复杂度比较高，营销 `活动+定价+配送` 输入环境信息高度耦合，需要引入类似multi-agent的协作机制。

举例：

当前商家满减档位是30-10，但客单价只有25，我应该推荐一个5元的加价购商品，还是建议调低满减门槛？

一种可能的实现框架：逻辑评估层 + 决策编排层

|              |                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------|
| <b>逻辑评估层</b> | 模型调用 <code>get_price_elasticity</code> 工具校验自己answer中关于“提价后订单量跌幅”的预估是否符合逻辑                                      |
| <b>决策编排层</b> | 配送费上涨2元，模型选择“增加满减额度”来对冲，还是选择“降低产品单价”？通过Rubric + Event Stream，定位出模型是在哪一步工具调用反馈导致了决策转向，定位到业务效果的关键影响因素，优化harness。 |

Openhands的优势是它从Event Stream可以完整捕获模型的thought字段，并且可以引入完全定制化的Rubric。这样，我们就可以定制一个 `metric collector`，专门用来评估模型在“定价策略”时的逻辑一致性。

## Appendix

### 1. 关于Claude Code修改system prompt的实现问题

| 方法                           | 是否真正修改<br>system prompt | 持久性      | 适用场景   |
|------------------------------|-------------------------|----------|--------|
| <code>--system-prompt</code> | 是                       | 单次会话     | 临时调整行为 |
| Output Styles                | 是（替换部分）                 | 保存到配置文件  | 长期工作模式 |
| tweakcc                      | 是（修改源码）                 | 更新后需重新应用 | 深度定制   |
| CLAUDE.md                    | 否（system-reminder）      | 文件保存     | 项目规范   |
| 环境变量                         | 是（简单模式）                 | 当前 shell | 极简使用   |

## Reference

- [1] [Best AI Agent Frameworks in 2026: Comparison + Decision Matrix](#)
- [2] <https://amirteymoori.com/dspy-3-build-evaluate-optimize-llm-pipelines/>
- [3] <https://docs.crewai.com/en/learn/customizing-agents>
- [4] <https://docs.openhands.dev/openhands/usage/developers/evaluation-harness>
- \*[5] [Effective harnesses for long-running agents](#)
- \*[6] [Harness design for long-running application development](#)
- \*[7] [Improving Deep Agents with harness engineering](#)
- [8] <https://docs.langchain.com/oss/python/deepagents/customization>